

Spring Boot from Scratch

A Friendly Journey from Zero to Production

Preface

This is not a list of facts. This is a journey.

I wrote this book for the developer who has heard about Spring Boot but feels lost the moment they open a tutorial. The official documentation is a 900-page reference. Most online tutorials throw code at you without context. Books often assume you already know Java EE.

This book is different. We will walk through Spring Boot like two friends having a long conversation. Each chapter builds on the last. By the end, you will not just *know* Spring Boot — you will be able to *build* a real, working backend project on your own.

A few promises:

- No academic jargon. Where complex words appear, they are explained immediately.
- Every concept gets a real-world example. We do not say "use this annotation" without explaining *why*.
- Code samples are short and complete. You can type them into your IDE and they will work.
- The chapters move from basics to advanced in a natural curve. Skip nothing on the first read.

Take your time. Brew some coffee. Open your IDE. Let us begin.

How to Read This Book

If you are completely new, read **Parts One through Five** in order. Type out the code yourself — do not copy and paste. After Part Five, you can start building real projects.

If you already know Spring basics and want only advanced topics, jump to **Part Eight**.

If you are preparing for an interview, scan the **Appendix** for the annotation reference and common error explanations.

There is one project that runs through the book — a small online bookstore. You will build it piece by piece. By the final chapter, it will be a complete, working application.

Table of Contents

Part One — The Beginning

- Chapter 1: A Story Before We Begin
- Chapter 2: Getting Your Tools Ready (Java, Maven, IDE)
- Chapter 3: Working Inside Eclipse, IntelliJ, and VS Code
- Chapter 4: Your First Spring Boot Project
- Chapter 5: A Walk Through the Project Folder

Part Two — How Spring Thinks

- Chapter 6: The Container That Holds Your Objects
- Chapter 7: Beans, Components, and Stereotypes
- Chapter 8: The Annotation Toolbox
- Chapter 9: Configuration Without Pain (properties, YAML, profiles)

Part Three — Building the Web

- Chapter 10: Your First REST API
- Chapter 11: Sending and Receiving Data
- Chapter 12: Validating Input
- Chapter 13: Handling Errors Gracefully
- Chapter 14: Talking to Other APIs

Part Four — Storing Data

- Chapter 15: Meeting the Database
- Chapter 16: Entities, Repositories, and the Magic of JPA
- Chapter 17: Relationships Between Tables
- Chapter 18: Transactions — Keeping Data Safe
- Chapter 19: When You Need More Power (JdbcTemplate, QueryDSL, custom queries)

Part Five — Locking the Door

- Chapter 20: A Gentle Introduction to Spring Security
- Chapter 21: JWT — Modern, Stateless Authentication
- Chapter 22: Roles, Permissions, and Method Security

Part Six — Becoming Production-Ready

- Chapter 23: Logs That Actually Help You
- Chapter 24: Testing Your Application (unit, integration, slice)
- Chapter 25: Monitoring with Actuator
- Chapter 26: Caching, Scheduling, and Async Work
- Chapter 27: Sending Emails, Handling Files

Part Seven — Shipping It

- Chapter 28: From JAR to Docker
- Chapter 29: Deployment Day

Part Eight — Going Further

- Chapter 30: Aspect-Oriented Programming
- Chapter 31: Events, Custom Starters, and Spring's Hidden Hooks
- Chapter 32: Real-Time — WebSockets and SSE
- Chapter 33: Reactive Programming with WebFlux
- Chapter 34: Spring Batch and GraphQL
- Chapter 35: Messaging with Kafka and RabbitMQ
- Chapter 36: Microservices with Spring Cloud

- Chapter 37: The Latest Spring Boot 3 Features

Appendix

- A: Common Errors and How to Fix Them
- B: The Bookstore Project Blueprint
- C: All Annotations Reference Card
- D: All Lombok Annotations
- E: Where Each Concept Is Used in Real Projects
- F: Final Checklist

PART ONE — THE BEGINNING

Chapter 1: A Story Before We Begin

Imagine you have been asked to build the backend for a bookstore. Users sign up, browse books, add them to a cart, and place orders. The backend must store data, send confirmation emails, talk to a payment provider, and remain stable when thousands of users hit it at once.

If you tried to build this in plain Java, you would write thousands of lines of plumbing. You would manually open database connections. You would write XML configuration files for the web server. You would tie yourself in knots trying to remember to close every resource.

This is the problem Spring Boot was born to solve.

Spring Boot is a Java framework that handles the boring plumbing for you. It opens database connections automatically. It runs a web server inside your application. It wires your objects together without asking you to manage them. Your job becomes simple — write the actual business logic of your bookstore, and let Spring Boot handle the rest.

A simple way to picture it: imagine Spring Boot as a fully equipped kitchen. The stove is already lit. The knives are sharpened. The ingredients are measured. You walk in, focus on the recipe, and cook. You do not have to chop firewood first.

Spring Boot did not appear out of thin air. It sits on top of a larger framework called **Spring**, which has been around since 2003. Spring itself is powerful but takes a lot of setup. Spring Boot looked at all that setup and said, "We can do this automatically." That is the magic. Spring is the engine; Spring Boot is the car built around it.

Today, every big Java backend you can think of runs on Spring Boot — Netflix, Amazon, Flipkart, Paytm, Swiggy, Zomato. When you place a food order on Swiggy, the backend service that receives your tap on the screen is almost certainly a Spring Boot application.

In the next chapter, we will get the tools you need to start writing your own Spring Boot apps.

Chapter 2: Getting Your Tools Ready

Before writing any code, we need three things on your computer.

The first is Java itself. Spring Boot 3 needs Java 17 or later. The simplest way to install it is to download a free distribution like Eclipse Temurin from <https://adoptium.net>. Pick Java 17 or Java 21, choose the installer for your operating system, run it, and click through. To check that it worked, open a terminal and type `java -version`. If it prints a version number starting with 17 or 21, you are ready.

The second is Maven. Maven is a build tool. It downloads the libraries your project uses and packages your code into a final file you can run. You will not see Maven much in your daily coding — your IDE runs it behind the scenes — but Spring Boot projects need it. Download Maven from <https://maven.apache.org>, unzip, add it to your PATH, and verify with `mvn -version`.

The third is an IDE. An IDE is an editor with superpowers for Java. The two best free options are **Eclipse** and **IntelliJ IDEA Community Edition**. **VS Code** also works with the right extensions. We will explore all three in the next chapter.

One more thing — install **Postman** from <https://www.postman.com>. Postman lets you send HTTP requests to your API and see the responses. We will use it constantly. There is also a free alternative called **Insomnia** if you prefer it.

That is your full toolkit. Java to run, Maven to build, an IDE to write, Postman to test.

Chapter 3: Working Inside Eclipse, IntelliJ, and VS Code

This chapter shows you how to work with Spring Boot inside the three most popular IDEs. Read the section for the IDE you chose; skip the others.

Eclipse — A Complete Walkthrough

Eclipse is free, mature, and used in many companies. To make it Spring-Boot-friendly, install the **Spring Tools 4** plugin. Open Eclipse, click `Help → Eclipse Marketplace`, search for "Spring Tools 4", click Install, and restart.

There is one extra step that often confuses beginners — **Lombok**. Lombok is a Java library that auto-generates getters, setters, and constructors using simple annotations like `@Data`. It will appear constantly in our code. Eclipse needs a special install for Lombok to work. Find a file called `lombok.jar` (download it from <https://projectlombok.org/download> or use the one Maven already downloaded inside `~/.m2/repository/org/projectlombok/`). Open a terminal and run `java -jar lombok.jar`. A window will pop up. Click **Specify location**, point it to your Eclipse installation folder, click **Install/Update**, and restart Eclipse. Without this step, Lombok annotations show as red errors in your code.

To open a Spring Boot project in Eclipse, click `File → Import → Maven → Existing Maven Projects`, browse to your project folder, and click Finish. Eclipse will spend a minute or two downloading dependencies. After that, you can right-click the project and choose `Run As → Spring Boot App`. The console will show `Started DemoApplication`, and your application is running.

To debug, click the gutter next to a line of code — a blue dot appears, marking a breakpoint. Right-click the project, choose `Debug As → Spring Boot App`. When execution reaches that line, Eclipse pauses and lets you inspect every variable.

A few shortcuts that will save you hours: **Ctrl+Shift+T** opens any class by name. **Ctrl+Shift+R** opens any file. **Ctrl+Shift+O** adds missing imports automatically. **Ctrl+1** offers quick fixes for red errors. **F3** jumps to the definition of whatever your cursor is on. **Ctrl+H** searches the whole project.

If you ever change `pom.xml` and Eclipse seems confused, right-click the project and select **Maven → Update Project**. That refreshes the dependency list.

IntelliJ IDEA — A Smoother Experience

IntelliJ IDEA Community Edition is also free and many developers prefer it for Spring Boot work. Open IntelliJ, click **File → Open**, and pick your project folder. IntelliJ detects Maven automatically and starts importing.

For Lombok to work in IntelliJ, you need to enable annotation processing. Open **Settings → Build, Execution, Deployment → Compiler → Annotation Processors** and tick **Enable annotation processing**. That is it.

To run, open your main class and click the green play button next to the `main` method. To debug, click the bug icon. Shortcuts: **Shift Shift** searches everything, **Ctrl+N** finds a class, **Alt+Enter** offers quick fixes.

IntelliJ Ultimate (paid) has deeper Spring Boot support — a special run config, a beans diagram, properties autocomplete. The Community edition is enough for learning.

VS Code — Lightweight but Capable

VS Code needs two extensions: **Extension Pack for Java** and **Spring Boot Extension Pack**. Install both from the Extensions sidebar. Then open your project folder. VS Code detects everything automatically.

To run, open the main class. A small "Run" link appears above the `main` method. Click it. To debug, click "Debug" next to it.

VS Code is faster to start than Eclipse or IntelliJ. The downside is fewer Spring-specific features. For deep work, Eclipse or IntelliJ remains better.

Chapter 4: Your First Spring Boot Project

Now we create our first project.

Open a browser and go to <https://start.spring.io>. This is the official Spring Boot project generator. Many developers — even senior ones — start every new project here.

The form on the left asks for some choices. Pick **Maven** as the project type. Pick **Java** as the language. Leave Spring Boot at the latest stable version (3.3.x at the time of writing). For Group, type `com.example`. For Artifact, type `bookstore`. Choose **Java 17**.

On the right is a list of dependencies. These are the building blocks of your project. Click "Add Dependencies" and add the following: **Spring Web** (for REST APIs), **Spring Data JPA** (for database access), **MySQL Driver** (we will use MySQL later, H2 first), **Lombok** (to reduce boilerplate), **Validation** (for input validation), and **Spring Boot DevTools** (for fast development reloads).

Click **Generate**. A ZIP file downloads. Unzip it. Open the folder in your IDE.

You now have your first Spring Boot project on disk.

When you open it, the IDE will spend a minute downloading the libraries listed inside `pom.xml`. After that, find the file called `BookstoreApplication.java`. Open it. You will see something like this:

```
package com.example.bookstore;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class BookstoreApplication {
    public static void main(String[] args) {
        SpringApplication.run(BookstoreApplication.class, args);
    }
}
```

Five lines of meaningful code. Yet when you run this class, a complete web server starts on your computer at port 8080. Magic? Not really — Spring Boot read the dependencies in your `pom.xml`, noticed `spring-boot-starter-web`, and quietly assembled a Tomcat web server inside the application.

The single annotation `@SpringBootApplication` does three jobs at once. First, it marks this class as a configuration class — meaning you could define beans (we will talk about beans soon) right inside it. Second, it switches on Spring Boot's **auto-configuration** — the magic that creates database connections, web servers, and JSON parsers based on what is in your classpath. Third, it scans the current package and every package below it for components, services, and controllers, so it can wire them together.

Run the application. In your IDE, find a green play button or right-click the main class and select **Run**. The console fills with text and ends with something like:

```
Started BookstoreApplication in 2.345 seconds (process running for 2.789)
```

Open a browser and visit `http://localhost:8080`. You will see a "Whitelabel Error Page" — because we have not built any endpoints yet. That is fine. The server is running. We will give it something to do in the next chapter.

Chapter 5: A Walk Through the Project Folder

Before we write code, let us understand where things live.

Inside your project folder, you have a `pom.xml` file at the root and a `src` folder. The `pom.xml` is Maven's instructions — it lists your dependencies and tells Maven how to build the project. Open it. You will see entries like `spring-boot-starter-web` and `spring-boot-starter-data-jpa`. These are called **starters** — pre-packaged bundles of libraries. Adding one starter gives you everything you need for that feature. The `spring-boot-starter-web` starter, for example, brings Spring MVC, Tomcat, JSON support, and validation — all from one line.

The `src` folder splits into two sub-folders. The `main` folder holds your application code. The `test` folder holds your tests. Inside `main` you will find `java` (your Java code) and `resources` (configuration files, templates, static files like CSS).

We will organize our Java code into clearly named packages. By convention, professional Spring Boot projects use a **layered architecture**:

- **controller** — receives HTTP requests and sends HTTP responses.
- **service** — contains the business logic. The brain of the application.
- **repository** — talks to the database.
- **entity** — Java classes that map to database tables.
- **dto** — small classes that travel between your application and the outside world.
- **config** — configuration classes.
- **exception** — your custom exceptions and the global error handler.

When a user clicks "Place Order" on the website, the request flows like this: it hits the **controller**, which calls the **service**, which calls the **repository**, which talks to the **database**. Then the response flows back. This separation is not a rule — it is wisdom learned from thousands of projects. Code organized this way is easier to test, easier to change, and easier to read.

Inside `src/main/resources`, you will find `application.properties`. This is where you put your settings — database URL, port number, log levels, anything that should be easy to change without recompiling. We will fill this file as we go.

You now know enough to start building. Let us go.

PART TWO — HOW SPRING THINKS

Chapter 6: The Container That Holds Your Objects

If you have written Java before, you are used to creating objects with `new`. You write `UserService u = new UserService();`, and the object is yours. You manage its life. You decide who can use it.

Spring works differently. Spring runs a thing called a **container** (sometimes called the **ApplicationContext**). The container is like a giant bag of objects. When your application starts, Spring fills this bag with all the objects you have declared. Then, when one part of your application needs another part, Spring reaches into the bag and hands it over.

You never write `new UserService()` in Spring code. You ask Spring for a `UserService`, and Spring gives you the one it already created.

The fancy term for this is **Inversion of Control**, or **IoC**. The word *inversion* sounds intimidating but the idea is simple. In normal Java, you control object creation. In Spring, you have *inverted* that control — Spring controls it, you just use the objects. Hence "inversion of control."

Here is the same idea in a cooking analogy. Imagine you are a chef in a fancy restaurant. In a normal kitchen, you walk to the storeroom yourself, pick the vegetables, chop them, and start cooking. That is plain Java. In a Spring kitchen, the prep cook has already chopped the vegetables and placed them next to your station

before you arrived. You do not even think about chopping — you just cook. The prep cook is Spring. The chopped vegetables are the objects in the container.

The way Spring delivers objects to your code is called **Dependency Injection**, or **DI**. "Dependency" here just means "an object that another object needs." When `OrderService` needs `PaymentService` to do its job, `PaymentService` is `OrderService`'s dependency. Spring *injects* that dependency.

There are three ways to receive an injected dependency. Two are common, one should be avoided.

The best way is **constructor injection**. You declare the dependency as a constructor parameter, and Spring passes it in when it creates the object.

```
@Service
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Notice the `final` keyword on the field. That means the field can only be assigned once — in the constructor. After that, no one can change it. That gives you safety: the dependency cannot be swapped at runtime by some misbehaving code.

The second way is **setter injection**, where Spring calls a setter method to provide the dependency. This is occasionally useful when the dependency is optional, but most modern code uses constructor injection.

The third way is **field injection**, where you write `@Autowired` directly on a field. It looks tidy but causes problems. You cannot make the field `final`. You cannot test the class easily because there is no constructor to pass mocks into. Many teams ban field injection from their codebase. We will not use it either.

When you see `@Service` on the class above, that tells Spring "please put this class into your container." Spring sees it during startup, creates one instance of `OrderService`, and stores it. Later, when something asks for an `OrderService`, Spring hands over that single instance.

You may wonder — does Spring create a new `OrderService` every time someone needs one? By default, no. Spring creates **one** instance per type and shares it with everyone. This default is called a **singleton scope**. You can change the scope if you need a fresh object each time, but for most service and repository classes, singleton is exactly what you want.

There are other scopes. **Prototype** scope creates a new instance every time. **Request** scope creates one instance per HTTP request, which is sometimes useful for tracking things specific to a single user's request. **Session** scope creates one per user session. You will rarely change scopes — but it is good to know they exist.

Each bean also has a **lifecycle**. When Spring creates the bean, you can run code at that moment using a method marked `@PostConstruct`. When Spring shuts down, you can run cleanup code with `@PreDestroy`.

These hooks are useful for things like pre-loading a cache from the database, or closing an open file before exit.

```
@Component
public class CacheLoader {

    @PostConstruct
    public void warmUp() {
        // load countries, currencies, anything that rarely changes
    }

    @PreDestroy
    public void cleanUp() {
        // close files, flush queues
    }
}
```

You do not call these methods yourself. Spring calls them at the right moment.

Chapter 7: Beans, Components, and Stereotypes

Spring calls every object it manages a **bean**. That word — *bean* — has stuck around since the late 1990s, but it just means "an object Spring takes care of."

You create beans by adding an annotation to a class. The most general one is `@Component`. When Spring scans your packages and finds a class marked `@Component`, it creates a bean.

But Java code has different kinds of classes. A class that does business logic is different from a class that talks to the database, which is different from a class that handles HTTP requests. To make your intent clear, Spring offers **specialized stereotypes**:

- `@Service` — marks a class that contains business logic.
- `@Repository` — marks a class that talks to the database. Spring adds extra exception translation here, turning database-specific errors into Spring's own exceptions.
- `@Controller` — marks a class that handles HTTP requests and returns HTML.
- `@RestController` — marks a class that handles HTTP requests and returns JSON.

All four are technically `@Component` underneath. The difference is just **clarity of intent**. When another developer opens your code, `@Service` tells them at a glance "this is business logic, not a controller."

Sometimes you need to create a bean from a class you did not write — say, a third-party library's class. In that case, you write a configuration method:

```
@Configuration
public class AppConfig {

    @Bean
    public RestTemplate restTemplate() {
```

```
        return new RestTemplate();  
    }  
}
```

The `@Configuration` annotation tells Spring this class contains bean definitions. The `@Bean` annotation says "the return value of this method is a bean, please add it to the container." Now you can inject a `RestTemplate` anywhere in your code.

What happens when two beans of the same type exist? Spring would not know which one to give you. You can mark one as the default with `@Primary`, or pick a specific one with `@Qualifier("name")`. We will see this when we get to advanced configuration.

Chapter 8: The Annotation Toolbox

Spring is full of annotations. They look intimidating at first — what does `@RestController` mean? what about `@RequestMapping`? — but each one stands for a small idea. Let us walk through the most common ones now, grouped by purpose, so you have a mental map. We will not memorize them. We will simply meet them. They will return throughout the book.

Annotations that mark a class as a bean include `@Component`, `@Service`, `@Repository`, `@Controller`, and `@RestController`. They tell Spring "include this class in your container."

Annotations for configuration include `@Configuration` for declaring a class of bean definitions, `@Bean` on a method whose return value is a bean, `@Value` for injecting a single property, and `@ConfigurationProperties` for injecting a whole group of properties.

Annotations that handle HTTP requests are `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`, and `@PatchMapping`. These map a URL and method to a Java function. Inside controller methods, `@PathVariable` captures part of the URL, `@RequestParam` captures a query string parameter, `@RequestBody` converts the JSON body of a request into a Java object, and `@RequestHeader` reads an HTTP header.

Annotations that connect to the database include `@Entity` (mark a class as a database table), `@Id` (the primary key field), `@GeneratedValue` (let the DB generate the id), and the relationship annotations `@OneToOne`, `@OneToMany`, `@ManyToOne`, `@ManyToMany`.

Annotations for validation include `@NotNull`, `@NotBlank`, `@Email`, `@Size`, `@Min`, `@Max`, and `@Pattern`. They check that user input is well-formed.

Annotations for transactions is mainly one — `@Transactional` — placed on a method to ensure that everything inside it succeeds together or rolls back together.

Annotations for testing include `@SpringBootTest`, `@WebMvcTest`, `@DataJpaTest`, `@MockBean`, and `@Test`.

We will meet every one of these in their natural setting. There is no need to memorize the list now.

Chapter 9: Configuration Without Pain

A real application needs settings. Database URL. Server port. API keys. Things that change between development and production.

Spring Boot reads those settings from a file called `application.properties` (or its YAML cousin, `application.yml`) located inside `src/main/resources`. When you start the app, Spring loads this file automatically.

A simple `application.properties` looks like this:

```
server.port=8081
spring.application.name=bookstore

spring.datasource.url=jdbc:mysql://localhost:3306/bookstore
spring.datasource.username=root
spring.datasource.password=secret
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

logging.level.org.springframework=INFO
```

Each line is a setting. `server.port=8081` makes the app listen on port 8081 instead of the default 8080. `spring.datasource.url` tells Spring where to find the database. `spring.jpa.hibernate.ddl-auto=update` tells Hibernate (the library that maps Java objects to database tables) to update the database schema automatically when your entities change — useful for development, dangerous for production.

YAML is the same idea written differently. Some developers find it cleaner:

```
server:
  port: 8081
spring:
  application:
    name: bookstore
  datasource:
    url: jdbc:mysql://localhost:3306/bookstore
    username: root
    password: secret
  jpa:
    hibernate:
      ddl-auto: update
```

Whichever style you choose, the meaning is identical.

To read a single property from your Java code, use `@Value`:

```
@Value("${app.discount:10}")
private int defaultDiscount;
```

That reads the property `app.discount` and stores it in `defaultDiscount`. The `:10` part is a default — if `app.discount` is not set, the variable becomes 10.

If you have many related properties, group them into a class with `@ConfigurationProperties`:

```
@ConfigurationProperties(prefix = "app")
@Component
@Data
public class AppProperties {
    private String name;
    private int discount;
    private List<String> admins;
}
```

Spring reads all `app.*` properties and fills the matching fields. So `app.name=Bookstore` becomes the `name` field, `app.discount=15` becomes `discount`, and so on. This approach is type-safe, easy to test, and supports nested objects.

Profiles

What if you want different settings for development versus production? That is what **profiles** are for. Create a file called `application-dev.properties` and another called `application-prod.properties`. When you start the app, you choose which one to activate by setting `spring.profiles.active=dev` (or `prod`). Spring loads `application.properties` first as a base, then layers the active profile's file on top.

You can even mark whole beans as belonging to a profile:

```
@Service
@Profile("dev")
public class FakePaymentGateway implements PaymentGateway { }

@Service
@Profile("prod")
public class RealPaymentGateway implements PaymentGateway { }
```

In development, you get the fake gateway — no real money changes hands. In production, the real one. Same interface, different behavior, switched by a single property.

A Note on Secrets

Never write real passwords or API keys into `application.properties` and commit it to Git. Anyone with access to the repository can read them.

Instead, reference an environment variable:

```
spring.datasource.password=${DB_PASSWORD}
```

Set `DB_PASSWORD` in your operating system or your deployment platform. In Kubernetes or AWS, this is handled by secret stores. We will return to this when we discuss deployment.

PART THREE — BUILDING THE WEB

Chapter 10: Your First REST API

We are ready to write real code. Let us build the first slice of our bookstore — a way to add, view, update, and delete books.

In modern web development, the backend usually does not return HTML pages. It returns **JSON**. A mobile app or a JavaScript frontend takes that JSON and decides how to display it. APIs that work this way are called **REST APIs**. The "REST" part stands for some elegant ideas about how URLs and HTTP methods should be used — but the short version is: GET to read, POST to create, PUT to update, DELETE to remove.

Let us create three files: an entity, a repository, and a controller.

The **entity** is a Java class that represents a row in a database table. Each field is a column. Spring's JPA layer reads this class and figures out how to save it to the database.

```
package com.example.bookstore.book;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Table(name = "books")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;
    private String author;
    private Double price;
}
```

A lot is happening here. `@Entity` tells JPA, "this class is a database table." `@Table(name = "books")` names the table. The Lombok annotations `@Data`, `@NoArgsConstructor`, and `@AllArgsConstructor` generate getters, setters, both kinds of constructor, `equals`, `hashCode`, and `toString` — saving us about 50 lines of boilerplate.

`@Id` marks the primary key. `@GeneratedValue(strategy = GenerationType.IDENTITY)` tells the database to generate the ID itself (using auto-increment in MySQL). We use `Long` rather than `long` because a new,

unsaved entity has no ID — and only `Long` can hold `null`.

Next, the **repository**.

```
package com.example.bookstore.book;

import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface BookRepository extends JpaRepository<Book, Long> {

    List<Book> findByTitleContainingIgnoreCase(String title);
}
```

Notice this is an **interface**, not a class. We never write the implementation. Spring Data JPA generates it for us at runtime. By extending `JpaRepository<Book, Long>`, we get a long list of free methods — `save`, `findById`, `findAll`, `deleteById`, `count`, paging, sorting — without writing a single line.

The line `List<Book> findByTitleContainingIgnoreCase(String title)` is even cleverer. Spring parses the method name and writes SQL automatically. `findBy` becomes `SELECT WHERE`, `Title` is the column, `Containing` becomes `LIKE '%...%'`, and `IgnoreCase` makes it case-insensitive. Calling `findByTitleContainingIgnoreCase("spring")` will return every book with "spring" anywhere in its title, regardless of capitalization.

Now the **service**.

```
package com.example.bookstore.book;

import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
@RequiredArgsConstructor
public class BookService {

    private final BookRepository repo;

    public List<Book> getAll() {
        return repo.findAll();
    }

    public Book get(Long id) {
        return repo.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Book " + id + "
not found"));
    }

    public Book create(Book b) {
        return repo.save(b);
    }
}
```

```
}

public Book update(Long id, Book b) {
    Book existing = get(id);
    existing.setTitle(b.getTitle());
    existing.setAuthor(b.getAuthor());
    existing.setPrice(b.getPrice());
    return repo.save(existing);
}

public void delete(Long id) {
    repo.deleteById(id);
}
}
```

The service holds the business logic. It calls the repository for data. The `@RequiredArgsConstructor` from Lombok creates a constructor that takes all `final` fields — in our case, `BookRepository` — so Spring can inject it. The `ResourceNotFoundException` is a custom class we will create in a moment. When the user asks for a book that does not exist, we throw it instead of returning null.

Finally, the **controller** — the doorway between HTTP and our code.

```
package com.example.bookstore.book;

import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/books")
@RequiredArgsConstructor
public class BookController {

    private final BookService service;

    @GetMapping
    public List<Book> all() {
        return service.getAll();
    }

    @GetMapping("/{id}")
    public Book one(@PathVariable Long id) {
        return service.get(id);
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Book create(@Valid @RequestBody Book b) {
        return service.create(b);
    }
}
```

```
    }

    @PutMapping("/{id}")
    public Book update(@PathVariable Long id, @RequestBody Book b) {
        return service.update(id, b);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id) {
        service.delete(id);
    }
}
```

`@RestController` says "this class handles HTTP requests and returns JSON."

`@RequestMapping("/api/books")` sets the base URL — every method here lives under `/api/books`.

`@GetMapping` without a path means the literal URL `/api/books`. `@GetMapping("/{id}")` matches URLs like `/api/books/5`, with `5` captured into the `id` parameter thanks to `@PathVariable`.

For creation, `@PostMapping` handles `POST /api/books`, and `@RequestBody` converts the incoming JSON into a `Book` object. `@ResponseStatus(HttpStatus.CREATED)` makes Spring return HTTP 201, the standard "created" code, instead of the default 200. `@Valid` tells Spring to run any validation annotations defined on the `Book` class before calling the method.

Add a simple H2 in-memory database to your `pom.xml` (or use MySQL if you already have it installed), then run the application. Open Postman, send `POST http://localhost:8080/api/books` with a JSON body like `{ "title": "Spring in Action", "author": "Craig Walls", "price": 39.99 }`, and you will see the book come back with an ID. Send `GET http://localhost:8080/api/books` and you get a list. You have built your first REST API.

Chapter 11: Sending and Receiving Data

Real APIs need more nuance than what we just wrote. We need control over response codes. We need to send back error objects, not just successes. We need to separate what the database stores from what the API exposes.

The first tool is `ResponseEntity`. It wraps your response and lets you set the status code, headers, and body independently:

```
@PostMapping
public ResponseEntity<Book> create(@RequestBody Book b) {
    Book saved = service.create(b);
    return ResponseEntity
        .status(HttpStatus.CREATED)
        .header("X-Created-Id", saved.getId().toString())
        .body(saved);
}
```


The second tool is the **DTO** — short for **Data Transfer Object**. Up to now, our controller has accepted a `Book` entity directly and returned one too. That works, but it is risky. Suppose later we add a `Book` field called `internalCostPrice`. We did not mean to expose it to API users, but if we still return raw `Book` objects, that field leaks into the JSON.

A DTO is a clean, intentional class for what your API accepts and returns. You decide exactly which fields are exposed. Here is a DTO using Java's record feature (a short way to define an immutable data class):

```
public record BookDto(String title, String author, Double price) {}
```

In the controller you accept a `BookDto` and convert it to a `Book` before saving:

```
@PostMapping
public Book create(@Valid @RequestBody BookDto dto) {
    Book b = new Book(null, dto.title(), dto.author(), dto.price());
    return service.create(b);
}
```

In a small project you might convert by hand. In a larger one, use a library called **MapStruct** that generates the conversion code at compile time — fast, type-safe, no reflection.

There are several ways to receive data in a controller method. `@PathVariable` captures URL segments. `@RequestParam` captures query string parameters such as `/books?author=alice`. `@RequestHeader` reads HTTP headers. `@CookieValue` reads cookies. They all work the same way — declare them as parameters and Spring fills them in.

```
@GetMapping
public List<Book> search(@RequestParam(required = false) String author,
                        @RequestParam(defaultValue = "0") int page) {
    // ...
}
```

`required = false` makes the parameter optional. `defaultValue = "0"` provides a default. Without these, missing parameters would cause an error.

Chapter 12: Validating Input

Imagine a user sends `POST /api/books` with an empty title. Or a negative price. Or a title that is 50,000 characters long. Without validation, your code accepts the garbage and saves it. That is a recipe for trouble — corrupted data, security holes, broken UIs.

Spring's solution is **Bean Validation**. You add annotations to your DTO that describe the rules:

```
public class BookDto {  
  
    @NotBlank(message = "Title is required")  
    @Size(max = 200, message = "Title too long")  
    private String title;  
  
    @NotBlank  
    private String author;  
  
    @NotNull  
    @Positive  
    private Double price;  
}
```

`@NotBlank` means "not null, not empty, not just whitespace." `@Size(max = 200)` limits the length. `@NotNull` ensures the field exists. `@Positive` rejects zero and negative numbers. There are many more — `@Email` for email format, `@Pattern(regexp = "...")` for any regex, `@Min` and `@Max` for numeric ranges, `@Past` and `@Future` for dates.

To activate validation, add `@Valid` to the controller parameter:

```
@PostMapping  
public Book create(@Valid @RequestBody BookDto dto) { ... }
```

Without `@Valid`, the annotations are completely ignored.

If validation fails, Spring throws a `MethodArgumentNotValidException`. By default that produces an ugly stack trace. In the next chapter we will catch it and turn it into a clean error response.

For more complex rules — say, "passwords must match each other" — you can write a custom validator. Define your own annotation like `@PasswordMatch`, write a `ConstraintValidator` class to enforce the rule, and Spring picks it up automatically. We will return to custom validators in the advanced section.

Chapter 13: Handling Errors Gracefully

In our service, when a book is not found, we throw `ResourceNotFoundException`. Right now that exception bubbles up and produces a 500 Internal Server Error with a stack trace. That is bad — the client sees Java internals, and they cannot tell the difference between "not found" and "server broken."

The cure is a **global exception handler**. One class catches every exception your application throws and returns a clean response.

```
@RestControllerAdvice  
public class GlobalExceptionHandler {  
  
    @ExceptionHandler(ResourceNotFoundException.class)
```

```

public ResponseEntity<ApiError> notFound(ResourceNotFoundException ex) {
    return ResponseEntity.status(HttpStatus.NOT_FOUND)
        .body(new ApiError(404, ex.getMessage()));
}

@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<Map<String, String>>
validation(MethodArgumentNotValidException ex) {
    Map<String, String> errors = new HashMap<>();
    ex.getBindingResult().getFieldErrors()
        .forEach(e -> errors.put(e.getField(), e.getDefaultMessage()));
    return ResponseEntity.badRequest().body(errors);
}

@ExceptionHandler(Exception.class)
public ResponseEntity<ApiError> generic(Exception ex) {
    return ResponseEntity.status(500).body(new ApiError(500, "Internal server
error"));
}

record ApiError(int status, String message) {}

```

`@RestControllerAdvice` tells Spring "watch every controller; if any of them throws an exception that I have a handler for, call my handler instead of producing a 500." For `ResourceNotFoundException`, we return 404. For validation errors, we build a clean map of field-to-message. For anything else, we return a generic 500 — but at least we hide the stack trace.

The `ResourceNotFoundException` class is just a simple subclass of `RuntimeException`:

```

public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String msg) { super(msg); }
}

```

Spring Boot 3 (built on Spring 6) introduced an even cleaner pattern called **ProblemDetail** — a standard JSON error format defined by an internet standard called RFC 7807. Use it if your team agrees:

```

@ExceptionHandler(ResourceNotFoundException.class)
public ProblemDetail notFound(ResourceNotFoundException ex) {
    ProblemDetail p = ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND,
ex.getMessage());
    p.setTitle("Resource Not Found");
    p.setProperty("timestamp", Instant.now());
    return p;
}

```

Whichever pattern you use, the principle stays the same: never let raw exceptions reach the client. Always return a clean, predictable error structure.

Chapter 14: Talking to Other APIs

A real application rarely lives alone. It calls a payment gateway, a maps service, a notification provider, or another team's microservice. To make HTTP calls from inside your Spring Boot app, you have three main tools.

The oldest is **RestTemplate**. Simple, synchronous, and still common in legacy code:

```
RestTemplate rt = new RestTemplate();
User u = rt.getForObject("https://api.example.com/users/1", User.class);
```

The modern replacement is **WebClient**, which can work asynchronously and supports streaming. It is part of the reactive package but works fine in regular projects too:

```
WebClient client = WebClient.create("https://api.example.com");
User u = client.get()
    .uri("/users/1")
    .retrieve()
    .bodyToMono(User.class)
    .block();
```

Spring 6.1 brought a new option called **RestClient** that combines RestTemplate's simplicity with WebClient's modern API. For brand new code, prefer this:

```
RestClient client = RestClient.create("https://api.example.com");
User u = client.get().uri("/users/{id}", 1).retrieve().body(User.class);
```

For calling other microservices in your own architecture, the favorite tool is **OpenFeign**. Instead of writing HTTP code, you declare an interface and Spring generates the implementation:

```
@FeignClient(name = "user-service", url = "http://user-service")
public interface UserClient {

    @GetMapping("/users/{id}")
    User get(@PathVariable Long id);
}
```

Inject `UserClient` like any bean and call `userClient.get(5L)`. Behind the scenes, Spring does the HTTP work for you. Add `@EnableFeignClients` on your main class to activate Feign.

Whenever you call an external service, things can go wrong — timeouts, 500 errors, network failures. We will handle those gracefully in the chapter on resilience.

PART FOUR — STORING DATA

Chapter 15: Meeting the Database

Most applications need to remember things. Users sign up — you must store them. Orders happen — you must record them. Spring Boot uses **JPA** (Java Persistence API) along with **Hibernate** (an implementation of JPA) to talk to relational databases like MySQL and PostgreSQL.

For learning, the friendliest database is **H2**. It runs entirely in memory inside your application — no install, no setup. Add this to `pom.xml`:

```
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
```

And to `application.properties`:

```
spring.datasource.url=jdbc:h2:mem:bookstore
spring.h2.console.enabled=true
```

Run the app and visit <http://localhost:8080/h2-console> — you get a tiny web UI to browse tables.

For real projects, you will use MySQL or PostgreSQL. For MySQL:

```
spring.datasource.url=jdbc:mysql://localhost:3306/bookstore?
createDatabaseIfNotExist=true
spring.datasource.username=root
spring.datasource.password=root
spring.jpa.hibernate.ddl-auto=update
```

The `ddl-auto` setting deserves attention. It controls how Hibernate manages the database schema:

- `none` — do nothing. The database must exist with all tables and columns. Safe for production.
- `validate` — check that tables match your entities; throw an error if not.
- `update` — add new columns automatically. Useful during development.
- `create` — drop everything and recreate every time the app starts. **All data lost** on every restart.
- `create-drop` — like `create`, but also drops everything on shutdown.

For learning, use `update`. For real production, use `none` or `validate` and manage schema changes with a tool called **Flyway** (we will visit Flyway later).

Behind the scenes, Spring Boot uses a connection pool called **HikariCP** — currently the fastest in the Java world. It opens a few database connections at startup and reuses them, because opening a fresh connection

every time is slow. You can tune it:

```
spring.datasource.hikari.maximum-pool-size=20
spring.datasource.hikari.minimum-idle=5
```

Most defaults are sensible. You only tune when you have proof that the defaults are not.

Chapter 16: Entities, Repositories, and the Magic of JPA

We have already built a `Book` entity and a `BookRepository`. Let us go deeper.

An entity describes a table. Spring Boot reads it and, with the help of Hibernate, generates SQL.

```
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 100)
    private String email;

    @Column(name = "full_name")
    private String fullName;

    @Enumerated(EnumType.STRING)
    private Role role;

    @CreationTimestamp
    private LocalDateTime createdAt;

    @UpdateTimestamp
    private LocalDateTime updatedAt;
}
```

Each line shapes the table. `@Column(nullable = false, unique = true, length = 100)` produces a column that cannot be null, must be unique across all rows, and stores up to 100 characters.

`@Enumerated(EnumType.STRING)` stores the enum value as its name ("ADMIN") rather than its ordinal number (which would break if you ever reordered the enum). `@CreationTimestamp` and `@UpdateTimestamp` are Hibernate annotations that fill those fields automatically.

The repository builds on this. When you extend `JpaRepository<User, Long>`, you immediately get methods you never wrote: `save`, `saveAll`, `findById`, `findAll`, `delete`, `deleteAll`, `count`, `existsById`, and more. You also get paging and sorting for free.

You can add more methods by following Spring Data's naming convention:

```
List<User> findByEmail(String email);
Optional<User> findByEmailAndActiveTrue(String email);
List<User> findByAgeGreaterThan(int age);
List<User> findByNameContainingIgnoreCaseOrderByAgeDesc(String name);
List<User> findTop5ByOrderByCreatedAtDesc();
long countByActive(boolean active);
```

Each method name is a recipe. Spring reads it and writes SQL. The keywords you can combine are extensive — *And, Or, Between, LessThan, GreaterThan, Like, Containing, StartingWith, EndingWith, In, NotIn, IsNull, IsNotNull, OrderBy, Top, First*. With enough practice, you will write surprisingly complex queries with method names alone.

For queries too complex to express in a name, write a JPQL query:

```
@Query("SELECT u FROM User u WHERE u.email = :email")
User searchByEmail(@Param("email") String email);
```

JPQL looks like SQL but operates on entity names, not table names. If you genuinely need raw SQL, use `nativeQuery = true`:

```
@Query(value = "SELECT * FROM users WHERE email = ?1", nativeQuery = true)
User byEmailRaw(String email);
```

For updates and deletes, add `@Modifying`:

```
@Modifying
@Transactional
@Query("UPDATE User u SET u.active = false WHERE u.id = :id")
void deactivate(@Param("id") Long id);
```

When you list many items, never return everything. Use **pagination**:

```
Page<Book> page = repo.findAll(PageRequest.of(0, 10,
Sort.by("price").descending()));
```

That returns the first ten books, sorted by price descending. The returned `Page` object also tells you the total count, total pages, whether there is a next page, and so on. In a controller you can accept a `Pageable` parameter directly, and clients can pass `?page=2&size=20&sort=price,desc`.

Chapter 17: Relationships Between Tables

Real data is connected. A user has many orders. An order has many items. A book has one author. JPA represents these connections with relationship annotations.

A **one-to-many** relationship is the most common. A user has many orders; each order has one user:

```
@Entity
public class User {

    @Id @GeneratedValue
    private Long id;

    @OneToMany(mappedBy = "user", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Order> orders = new ArrayList<>();
}

@Entity
public class Order {

    @Id @GeneratedValue
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id")
    private User user;
}
```

A few subtleties matter here. `mappedBy = "user"` tells Hibernate that the `Order` entity owns the relationship — Hibernate will use the foreign key column on the `orders` table instead of creating a separate join table. `cascade = CascadeType.ALL` means saving the user also saves any new orders attached to them. `orphanRemoval = true` means removing an order from `user.getOrders()` deletes it from the database. `fetch = FetchType.LAZY` is critical — without it, every time you load an `Order`, Hibernate also loads the `User`. With LAZY, the user is loaded only when you actually access it.

A common painful bug related to lazy loading is the **N+1 problem**. Suppose you fetch 100 orders and then iterate, calling `order.getUser().getName()` on each. Hibernate runs one query for the orders and then 100 more queries — one per order — to fetch each user. The fix is to eagerly fetch the user in the same query:

```
@Query("SELECT o FROM Order o JOIN FETCH o.user WHERE o.status = :s")
List<Order> findByStatus(@Param("s") String status);
```

Or use `@EntityGraph`:

```
@EntityGraph(attributePaths = {"user", "items"})
List<Order> findByStatus(String status);
```

A **one-to-one** relationship is simpler. One user has one address:


```
@OneToOne(cascade = CascadeType.ALL)
@JoinColumn(name = "address_id")
private Address address;
```

A **many-to-many** relationship requires a join table:

```
@ManyToMany
@JoinTable(
    name = "student_course",
    joinColumns = @JoinColumn(name = "student_id"),
    inverseJoinColumns = @JoinColumn(name = "course_id"))
private Set<Course> courses;
```

Use `Set` (not `List`) for many-to-many to avoid duplicates when the same pair appears twice.

There is also a special form called `@Embeddable`. When you have a set of fields that always travel together — like an address (street, city, zip) — you can group them into one class but still store them as columns in the parent table:

```
@Embeddable
public class Address {
    private String street;
    private String city;
    private String zip;
}

@Entity
public class User {
    @Embedded
    private Address address;
}
```

The `users` table now has columns `street`, `city`, `zip` directly. No extra table.

Chapter 18: Transactions — Keeping Data Safe

When money moves between two bank accounts, two things must happen together: the source debit and the destination credit. If the application crashes between them, you lose money — or worse, create it from thin air. The cure is a **transaction**. A transaction is a group of operations that either all succeed or all fail.

In Spring Boot, you mark a method `@Transactional`:

```
@Service
public class TransferService {
```

```
@Transactional
public void transfer(Long fromId, Long toId, BigDecimal amount) {
    Account from = repo.findById(fromId).orElseThrow();
    Account to = repo.findById(toId).orElseThrow();
    from.debit(amount);
    to.credit(amount);
    repo.save(from);
    repo.save(to);
}
```

If anything inside this method throws a `RuntimeException`, Spring rolls back every change made since the method began. Either both accounts are updated, or neither is.

A few rules to remember. First, put `@Transactional` on service methods, not on controllers or repositories. The service is where business operations live. Second, by default Spring only rolls back on `RuntimeException`. If you throw a checked `Exception`, you must declare `@Transactional(rollbackFor = Exception.class)`. Third, always use `BigDecimal` for money, never `double` — floating point loses precision in surprising ways.

Read-only operations can be marked `@Transactional(readOnly = true)`. This is a small performance hint to Hibernate.

There is a subtle topic called **propagation** — what happens when one transactional method calls another. The default, `REQUIRED`, joins the existing transaction. `REQUIRES_NEW` suspends the current transaction and starts a fresh, independent one — useful for things like audit logs that must persist even if the main transaction rolls back. We will revisit propagation in the advanced section.

For environments where two users might edit the same record at once, you can add **optimistic locking** with a `@Version` field:

```
@Entity
public class Product {
    @Id @GeneratedValue Long id;
    private String name;

    @Version
    private Long version;
}
```

Each update checks the version number. If two users started with version 3 and one of them saved first (bumping to 4), the second user's save fails — Hibernate throws an `OptimisticLockException`. You can catch it and ask the user to reload and try again. This prevents silent overwrites.

For tighter control, use **pessimistic locking** — actually locking the row in the database until your transaction commits. It blocks other readers or writers, so use it sparingly:

```
@Query("SELECT a FROM Account a WHERE a.id = :id")
@Lock(LockModeType.PESSIMISTIC_WRITE)
Account findForUpdate(@Param("id") Long id);
```

Chapter 19: When You Need More Power

Spring Data JPA is wonderful for ninety percent of database work. But sometimes you need raw power — a complex report query, a dynamic search form, or pure performance.

For raw SQL queries, drop down to `JdbcTemplate` or `NamedParameterJdbcTemplate`:

```
@Service
public class ReportService {

    private final JdbcTemplate jdbc;
    private final NamedParameterJdbcTemplate named;

    public ReportService(JdbcTemplate jdbc, NamedParameterJdbcTemplate named) {
        this.jdbc = jdbc;
        this.named = named;
    }

    public int countActive() {
        return jdbc.queryForObject(
            "SELECT COUNT(*) FROM users WHERE active = true", Integer.class);
    }

    public List<User> findByStatus(String status) {
        return named.query(
            "SELECT id, email FROM users WHERE status = :status",
            Map.of("status", status),
            (rs, n) -> new User(rs.getLong("id"), rs.getString("email"))
        );
    }
}
```

`JdbcTemplate` is faster than JPA for read-heavy reports because it skips the entity machinery.

For complex search forms — "find users whose name contains X, age greater than Y, registered after Z, with role R, but only if those filters were provided" — use **Specifications** (Spring Data's typed dynamic query builder) or **QueryDSL** (a third-party library that generates a type-safe query API from your entities).

Specifications come built in:

```
public interface UserRepository extends JpaRepository<User, Long>,
    JpaSpecificationExecutor<User> { }

public class UserSpecs {
```

```
public static Specification<User> hasName(String name) {
    return (root, q, cb) -> name == null ? null : cb.like(root.get("name"),
"%\" + name + \"%");
}
public static Specification<User> minAge(Integer age) {
    return (root, q, cb) -> age == null ? null : cb.ge(root.get("age"), age);
}
}

Specification<User> spec =
Specification.where(UserSpecs.hasName("alice")).and(UserSpecs.minAge(18));
List<User> users = repo.findAll(spec);
```

Spring assembles the SQL based on which filters were provided.

For production databases, never use `ddl-auto=update`. Instead, use **Flyway**:

```
<dependency>
  <groupId>org.flywaydb</groupId>
  <artifactId>flyway-core</artifactId>
</dependency>
```

Place SQL migration files in `src/main/resources/db/migration/`, named like `V1__create_users.sql`, `V2__add_email_column.sql`. Flyway runs them in order on startup and remembers which ones it has already run. This gives you version-controlled, reviewable, repeatable schema changes — essential for any team that ships software.

For projects where your data does not fit neatly into rows and columns — logs, content with flexible structure, social-media-like timelines — Spring Data also supports **MongoDB** (and other NoSQL stores). The pattern is nearly identical: an entity-like class marked `@Document`, and a repository interface extending `MongoRepository`.

PART FIVE — LOCKING THE DOOR

Chapter 20: A Gentle Introduction to Spring Security

The moment you add `spring-boot-starter-security` to your `pom.xml`, two things happen. Every endpoint in your application becomes protected. And Spring prints a generated password into the console on startup.

Try it. Visit `/api/books` after adding security — you get a 401 Unauthorized. Use the username `user` and the generated password as a Basic Auth header, and suddenly your books appear again.

This default behavior is the world's simplest authentication. Real applications need more. The first step is to write a `SecurityFilterChain` — a sequence of filters that processes every request and decides whether the user is allowed in.

```

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/public/**").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .httpBasic(Customizer.withDefaults());
        return http.build();
    }

    @Bean
    public PasswordEncoder encoder() {
        return new BCryptPasswordEncoder();
    }
}

```

Let us read this slowly. `csrf.disable()` switches off Cross-Site Request Forgery protection. CSRF protects browser sessions that use cookies; if your API is consumed by mobile apps or single-page apps using tokens, you do not need it. The `authorizeHttpRequests` block sets the rules — anyone can hit `/api/public/**`, only users with the ADMIN role can hit `/api/admin/**`, and everything else requires a logged-in user. `httpBasic` enables Basic Authentication for now.

The `PasswordEncoder` bean is critical. It uses **BCrypt**, an industry-standard hashing algorithm. When a user signs up, you hash their password with `encoder.encode(plain)` and store the hash. When they log in, you check with `encoder.matches(plain, hash)`. Never, ever store plain passwords in your database.

For Spring Security to recognize your application's users, write a class that implements `UserDetailsService`:

```

@Service
@RequiredArgsConstructor
public class UserDetailsServiceImpl implements UserDetailsService {

    private final UserRepository repo;

    @Override
    public UserDetails loadUserByUsername(String email) {
        User u = repo.findByEmail(email)
            .orElseThrow(() -> new UsernameNotFoundException(email));
        return org.springframework.security.core.userdetails.User.builder()
            .username(u.getEmail())
            .password(u.getPassword())
            .roles(u.getRole().name())
            .build();
    }
}

```

Authentication is "who are you?" — the login step. Authorization is "what are you allowed to do?" — the permission step. Spring Security handles both. Authentication checks the password. Authorization checks roles. The rules in `authorizeHttpRequests` are authorization rules.

Chapter 21: JWT — Modern, Stateless Authentication

Basic Authentication is fine for a tutorial, but real APIs use **JWT** — JSON Web Tokens. The idea is elegant. When the user logs in, you create a signed string that contains their identity and an expiry time. You hand the string to the client. From then on, the client attaches the string to every request in an **Authorization: Bearer <token>** header. Your server validates the signature on each request and grants access. No session in memory. The server can scale to a hundred instances and any of them can serve any request — because the token carries the proof.

The token has three parts separated by dots: header, payload, and signature. The header says "this is a JWT signed with HS256." The payload contains claims like the username and expiry. The signature is a hash of the first two parts using a secret key that only your server knows. Anyone can read the payload — it is just Base64-encoded JSON — but no one can fake a new token without the secret.

To generate a token:

```
public String generateToken(String username) {
    return Jwts.builder()
        .setSubject(username)
        .setIssuedAt(new Date())
        .setExpiration(new Date(System.currentTimeMillis() + 3_600_000))
        .signWith(Keys.hmacShaKeyFor(secret.getBytes()),
SignatureAlgorithm.HS256)
        .compact();
}
```

To check a token on every request, you write a **filter** that runs before the controller:

```
public class JwtFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse
res, FilterChain chain)
        throws ServletException, IOException {

        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
```

```
String username = jwtService.extractUsername(token);

UserDetails ud = userDetailsService.loadUserByUsername(username);
UsernamePasswordAuthenticationToken auth =
    new UsernamePasswordAuthenticationToken(ud, null,
ud.getAuthorities());
SecurityContextHolder.getContext().setAuthentication(auth);
    }
    chain.doFilter(req, res);
}
}
```

Wire the filter into your security chain and tell Spring not to keep server-side sessions:

```
http.addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
http.sessionManagement(sm ->
    sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS));
```

Now your API is stateless and JWT-based.

For a deeper option, your token can come from an external identity provider like Keycloak, Auth0, or Okta. In that case use the OAuth2 Resource Server starter — Spring fetches the provider's public keys and validates incoming tokens automatically. We will return to OAuth2 in the advanced section.

Chapter 22: Roles, Permissions, and Method Security

URL-based rules in `SecurityFilterChain` are coarse. Sometimes you want finer-grained control — say, "users can update their own profile, but admins can update anyone's." For that, use **method security**.

Enable it:

```
@EnableMethodSecurity
@Configuration
public class SecurityConfig { ... }
```

Then annotate your service or controller methods:

```
@PreAuthorize("hasRole('ADMIN')")
public void deleteProduct(Long id) { }

@PreAuthorize("#userId == authentication.principal.id")
public User getOwnProfile(Long userId) { }

@PostAuthorize("returnObject.owner == authentication.name")
public Document loadDoc(Long id) { }
```

The expressions inside `@PreAuthorize` are written in **SpEL** — Spring Expression Language. They can read method arguments (`#userId`), the current authentication (`authentication.principal.id`), and even the return value (in `@PostAuthorize`).

For finer audit and stronger security, add response security headers — HSTS, X-Frame-Options, Content Security Policy — Spring Security adds safe defaults automatically, but you can customize them in your chain.

PART SIX — BECOMING PRODUCTION-READY

Chapter 23: Logs That Actually Help You

Logs are the windows into a running application. When something breaks at 2 AM, the difference between a five-minute fix and a five-hour outage is whether your logs are useful.

Spring Boot ships with **SLF4J** as the API and **Logback** as the implementation. Lombok provides a handy annotation that adds a logger automatically:

```
@Slf4j
@Service
public class OrderService {

    public void place(Order o) {
        log.info("Placing order id={} user={}", o.getId(), o.getUserId());
        log.debug("Full order details: {}", o);
        try {
            // ...
        } catch (Exception ex) {
            log.error("Order failed", ex);
        }
    }
}
```

Note two important habits. First, always use placeholders (`{}`) instead of string concatenation (`+`). Placeholders are faster and only build the message if the log level is enabled. Second, always pass the exception as the last argument to `log.error` — that way the full stack trace is logged.

Log levels, from most chatty to least, are `trace`, `debug`, `info`, `warn`, `error`. In production you usually run at `info`. When debugging a specific class, you can crank up its level:

```
logging.level.root=INFO
logging.level.com.example.bookstore.order=DEBUG
logging.file.name=logs/app.log
```

In cloud environments, logs are typically read by tools like Elasticsearch or Datadog. They prefer **JSON-structured logs** so they can parse fields easily. Add the Logstash encoder and configure Logback to output JSON. We will visit this in the deployment chapter.

To trace a single request as it travels through your code, use the **MDC** (Mapped Diagnostic Context). You stash key-value pairs on the current thread, and Logback automatically includes them in every log line. With Micrometer Tracing, every incoming request gets a `traceId` placed into MDC, so all logs for that request can be grouped together — even across multiple microservices.

Chapter 24: Testing Your Application

Skipping tests feels productive in the moment and disastrous later. A Spring Boot project that is well-tested gives its team courage to refactor, to upgrade libraries, to chase performance. Without tests, every change is a gamble.

The world of Spring Boot tests has three layers.

Unit tests verify one class in isolation. They are tiny and fast. Use **Mockito** to fake the dependencies.

```
@ExtendWith(MockitoExtension.class)
class BookServiceTest {

    @Mock BookRepository repo;
    @InjectMocks BookService service;

    @Test
    void getById_found() {
        when(repo.findById(1L)).thenReturn(Optional.of(new Book(1L, "Spring",
" Walls", 39.99)));
        Book b = service.get(1L);
        assertEquals("Spring", b.getTitle());
    }

    @Test
    void getById_notFound() {
        when(repo.findById(99L)).thenReturn(Optional.empty());
        assertThrows(ResourceNotFoundException.class, () -> service.get(99L));
    }
}
```

`@Mock` creates a fake repository — calls to it return whatever you stub. `@InjectMocks` creates the real service under test and injects the mocks. We never touch the database.

Integration tests boot up the whole Spring context. They are slower but verify that pieces fit together.

`MockMvc` lets you fire requests at your controllers without starting a real server:

```
@SpringBootTest
@AutoConfigureMockMvc
class BookControllerTest {

    @Autowired MockMvc mvc;

    @Test
```

```
void getAll() throws Exception {
    mvc.perform(get("/api/books"))
        .andExpect(status().isOk());
}
```

Slice tests load only one layer. `@DataJpaTest` loads just JPA against an H2 database.

`@WebMvcTest(SomeController.class)` loads just MVC for one controller. `@JsonTest` loads just JSON serialization. Slice tests run faster than `@SpringBootTest` because they skip parts of the framework you do not need.

For database tests that should still run against a real database (not H2), use **Testcontainers**. It spins up a real MySQL or PostgreSQL in Docker for the duration of the test:

```
@SpringBootTest
@Testcontainers
class OrderIntegrationTest {

    @Container
    @ServiceConnection
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>
("postgres:16");
}
```

The `@ServiceConnection` annotation (introduced in Spring Boot 3.1) auto-wires the container's connection details into Spring — no manual property setup required.

When you need to simulate a logged-in user during a security test, use `@WithMockUser`:

```
@Test
@WithMockUser(roles = "ADMIN")
void admin_can_delete_book() throws Exception {
    mvc.perform(delete("/api/books/1"))
        .andExpect(status().isNoContent());
}
```

To measure how thoroughly your tests cover the code, plug in **JaCoCo**. After `mvn test`, you get an HTML report listing which lines were and were not exercised. Most teams aim for at least 70 to 80 percent line coverage on business code.

Chapter 25: Monitoring with Actuator

A running application needs to answer questions like "are you alive?", "how much memory are you using?", and "what version of the code are you running?". The **Actuator** starter exposes a set of endpoints under `/actuator` that answer these questions.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

```
management.endpoints.web.exposure.include=health,info,metrics,loggers,prometheus
management.endpoint.health.show-details=always
```

`/actuator/health` returns **UP** or **DOWN**. Kubernetes calls this URL repeatedly to check whether to restart your pod. `/actuator/metrics` exposes detailed numbers — CPU, memory, request counts, database connection pool size. `/actuator/loggers` lets you change log levels at runtime without restarting.

You can write your own health indicators. Suppose your app depends on Kafka — write a check:

```
@Component
public class KafkaHealthIndicator implements HealthIndicator {

    @Override
    public Health health() {
        try {
            // ping kafka
            return Health.up().build();
        } catch (Exception e) {
            return Health.down().withException(e).build();
        }
    }
}
```

Now `/actuator/health` reports Kafka too.

For metrics, Spring Boot integrates with **Micrometer**, which can ship metrics to Prometheus, Datadog, New Relic, and many others. Add the Prometheus dependency, and `/actuator/prometheus` becomes a scrape target for Prometheus servers. Grafana dashboards visualize the data.

For multi-service deployments, **Spring Boot Admin** is a small dashboard that shows the health and metrics of every service in one UI. Useful for teams managing a dozen microservices.

Chapter 26: Caching, Scheduling, and Async Work

Three small features make a big difference in production performance.

Caching stores expensive results so you do not compute them twice. Annotate the slow method and Spring handles the rest.

```
@SpringBootApplication
@EnableCaching
public class App { }

@Service
public class CountryService {

    @Cacheable("countries")
    public List<Country> all() {
        // slow DB call, runs only the first time
    }

    @CacheEvict(value = "countries", allEntries = true)
    public void refresh() { }
}
```

The default cache is in memory and dies with the app. For a shared cache across multiple app instances, point Spring at Redis:

```
spring.cache.type=redis
spring.data.redis.host=localhost
spring.data.redis.port=6379
```

Scheduling runs code on a timer. Maybe you want to send a daily report at 2 AM, or clean up expired sessions every five minutes.

```
@SpringBootApplication
@EnableScheduling
public class App { }

@Component
public class Jobs {

    @Scheduled(fixedRate = 60_000)
    public void heartbeat() { }

    @Scheduled(cron = "0 0 2 * * *")
    public void nightlyCleanup() { }
}
```

The cron expression has six fields: second, minute, hour, day of month, month, and day of week. Many developers find cron tricky — use a cron generator the first few times.

When you run more than one instance of your app, scheduled tasks fire on every instance. Often you want only one instance to do the work. **ShedLock** solves this — it uses a database row to coordinate which instance "owns" the lock.

Async work runs a method in a background thread so the caller does not wait. When a user places an order, you do not want them staring at a loading spinner while the confirmation email is sent. Send it asynchronously.

```
@SpringBootApplication
@EnableAsync
public class App { }

@Service
public class MailService {

    @Async
    public CompletableFuture<Void> sendInvoice(String to) {
        // runs on a background thread
        return CompletableFuture.completedFuture(null);
    }
}
```

By default, Spring uses a simple executor for async tasks. For production, configure a real thread pool:

```
@Configuration
@EnableAsync
public class AsyncConfig {

    @Bean(name = "taskExecutor")
    public Executor taskExecutor() {
        ThreadPoolTaskExecutor ex = new ThreadPoolTaskExecutor();
        ex.setCorePoolSize(5);
        ex.setMaxPoolSize(20);
        ex.setQueueCapacity(100);
        ex.setThreadNamePrefix("async-");
        ex.initialize();
        return ex;
    }
}
```

You can also retry failed operations automatically. Add **Spring Retry**, annotate the method `@Retryable`, and the same call is attempted multiple times with backoff.

Chapter 27: Sending Emails, Handling Files

Backend applications often need to send email — order confirmations, password resets, reports. Spring's mail starter makes it simple.

```
<dependency>
<groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-starter-mail</artifactId>
</dependency>
```

```
spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=you@gmail.com
spring.mail.password=app-password
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true
```

```
@Service
@RequiredArgsConstructor
public class EmailService {

    private final JavaMailSender sender;

    public void send(String to, String subject, String body) {
        SimpleMailMessage msg = new SimpleMailMessage();
        msg.setTo(to);
        msg.setSubject(subject);
        msg.setText(body);
        sender.send(msg);
    }
}
```

For HTML emails with attachments, use `MimeMessageHelper`. Combine with `@Async` so the user does not wait for the SMTP server.

File uploads use `MultipartFile`:

```
@PostMapping("/upload")
public String upload(@RequestParam("file") MultipartFile file) throws IOException
{
    Path p = Paths.get("uploads/" + file.getOriginalFilename());
    Files.write(p, file.getBytes());
    return "Uploaded: " + p;
}
```

In production, do not save files on the local disk — the disk dies, the file is lost. Save to cloud storage like AWS S3 instead.

For downloads, set the right header so the browser saves the file:

```
@GetMapping("/download/{name}")
public ResponseEntity<Resource> download(@PathVariable String name) throws
```

```
IOException {
    Path p = Paths.get("uploads/" + name);
    Resource r = new UrlResource(p.toUri());
    return ResponseEntity.ok()
        .header(HttpHeaders.CONTENT_DISPOSITION, "attachment; filename=\"" +
name + "\"")
        .body(r);
}
```

Adjust upload size limits in properties:

```
spring.servlet.multipart.max-file-size=10MB
spring.servlet.multipart.max-request-size=10MB
```

PART SEVEN — SHIPPING IT

Chapter 28: From JAR to Docker

When you finish coding, you must hand off something runnable. Spring Boot produces a **fat JAR** — a single file containing your code, every library, and an embedded Tomcat server. Run it like this:

```
mvn clean package
java -jar target/bookstore-0.0.1-SNAPSHOT.jar
```

That is enough to run on a server. But modern deployment usually uses **Docker** — a way to bundle your application together with the operating system bits it needs into a portable container.

A minimal **Dockerfile**:

```
FROM eclipse-temurin:17-jdk-alpine
WORKDIR /app
COPY target/bookstore-0.0.1-SNAPSHOT.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Run:

```
docker build -t bookstore .
docker run -p 8080:8080 bookstore
```

For a smaller, more secure image, use a **multi-stage build** — compile inside one image and copy the result into a tiny runtime image:

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /src
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src ./src
RUN mvn -q -DskipTests package

FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=build /src/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

You can skip writing a Dockerfile entirely with Spring Boot's built-in buildpacks support:

```
mvn spring-boot:build-image
```

That produces an OCI image automatically, with sensible defaults.

When your app needs a database to run with, **docker-compose** brings both up together:

```
version: "3"
services:
  db:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: bookstore
    ports: ["3306:3306"]
  app:
    build: .
    depends_on: [db]
    ports: ["8080:8080"]
    environment:
      SPRING_DATASOURCE_URL: jdbc:mysql://db:3306/bookstore
      SPRING_DATASOURCE_USERNAME: root
      SPRING_DATASOURCE_PASSWORD: root
```

Run `docker compose up --build` and everything starts. Inside Docker, services find each other by service name — that is why our URL is `jdbc:mysql://db:3306/bookstore`.

Spring Boot 3.1 added **Docker Compose support** directly — drop a `compose.yaml` in your project root, add `spring-boot-docker-compose` as a runtime dependency, and Spring Boot will start the compose stack automatically when you run the app. Wonderful for local development.

Chapter 29: Deployment Day

Where do you actually run your container? Several options.

A small project might run on a single virtual server — buy an EC2 instance, install Docker, run your image, point a domain at the IP. Fine for hobby projects and prototypes.

A serious project uses **Kubernetes**, an orchestrator that manages your containers across many machines. Each microservice becomes a *Deployment*, and a *Service* exposes it inside the cluster. Spring Boot's Actuator probes — `/actuator/health/liveness` and `/actuator/health/readiness` — tell Kubernetes whether a pod is alive and ready to receive traffic. When you push a new version, Kubernetes does a rolling deploy — gradually replacing old pods with new ones with zero downtime, provided your application supports **graceful shutdown**:

```
server.shutdown=graceful
spring.lifecycle.timeout-per-shutdown-phase=30s
```

With graceful shutdown enabled, when Kubernetes sends a SIGTERM signal, Spring Boot stops accepting new requests but finishes the in-flight ones before exiting.

Many companies use platforms that sit on top of Kubernetes and hide most of its complexity — AWS Fargate, Google Cloud Run, Heroku, Render, Railway, Fly.io. You hand them an image, they handle the orchestration.

Whatever platform you use, follow the **12-Factor App** principles. Store config in environment variables, not in files. Log to standard output, not to files. Treat backing services like databases as plug-in resources. Each instance should be stateless. Restart fast and shut down clean. These principles came from Heroku in 2011 and have aged remarkably well.

A **CI/CD pipeline** automates everything from commit to deployment. A typical GitHub Actions workflow:

```
name: build
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: 'temurin', java-version: '17' }
      - run: mvn -B verify
      - run: mvn -B spring-boot:build-image
```

Every push to `main` runs your tests, builds an image, and pushes it to a registry. Add a deployment step and you have a fully automated pipeline. Many companies enforce that no human can deploy by hand — everything must go through the pipeline.

PART EIGHT — GOING FURTHER

Chapter 30: Aspect-Oriented Programming

Some concerns spread across your whole codebase. Logging every service call. Measuring how long methods take. Auditing who did what. Adding these features by hand means scattering the same six lines of code throughout your project — and breaking a sacred rule: *do not repeat yourself*.

Aspect-Oriented Programming, or **AOP**, lets you add behavior to many methods at once without touching their source. Add the AOP starter:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

Write an aspect:

```
@Aspect
@Component
@Slf4j
public class LogAspect {

    @Around("execution(* com.example..service..*(..))")
    public Object log(ProceedingJoinPoint pjp) throws Throwable {
        long start = System.currentTimeMillis();
        Object result = pjp.proceed();
        log.info("{} took {} ms", pjp.getSignature(), System.currentTimeMillis() -
start);
        return result;
    }
}
```

The expression `execution(* com.example..service..*(..))` selects every method in every class inside any package below `com.example..service...`. `pjp.proceed()` actually runs the wrapped method; the lines before and after are your added behavior. The result: every service method now logs how long it took, and you did not edit a single service.

Real-world uses include audit logging, performance metrics, security checks, and even managing transactions (Spring's own `@Transactional` is implemented as an aspect under the hood).

Chapter 31: Events, Custom Starters, and Spring's Hidden Hooks

When two parts of your application need to react to the same event but should not depend on each other, use Spring's event system.

```

public class OrderPlacedEvent extends ApplicationEvent {
    private final Order order;
    public OrderPlacedEvent(Object src, Order o) { super(src); this.order = o; }
    public Order getOrder() { return order; }
}

@Service
@RequiredArgsConstructor
public class OrderService {

    private final ApplicationEventPublisher publisher;

    public void place(Order o) {
        // save order
        publisher.publishEvent(new OrderPlacedEvent(this, o));
    }
}

@Component
public class EmailListener {

    @EventListener
    @Async
    public void on(OrderPlacedEvent e) {
        // send confirmation email
    }
}

@Component
public class InventoryListener {

    @EventListener
    public void on(OrderPlacedEvent e) {
        // deduct stock
    }
}

```

`OrderService` does not know who is listening. The email service, inventory service, and a future audit service each plug in independently. Add `@Async` to a listener and it runs on a background thread. This pattern is the seed of event-driven design.

Spring also exposes hooks for its own lifecycle. `CommandLineRunner` and `ApplicationRunner` run code immediately after startup — perfect for seeding default data or warming caches. `ApplicationReadyEvent` fires when the app is fully ready; you might notify a monitoring system or post a Slack message at that point.

For very deep customization, you can write a **custom auto-configuration** that other projects use as a dependency. Your company's logging starter, for instance, could be a single artifact that, when added, configures structured JSON logging, MDC for trace IDs, and a custom log format — all automatically. The `@AutoConfiguration` annotation plus a file in `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` is all you need

to make a starter. Many large companies ship internal starters like this for everything from security to messaging.

Chapter 32: Real-Time — WebSockets and SSE

Most APIs are request-response — the client asks, the server answers. Some applications need more — a chat app, a live dashboard, a notification system. Two technologies serve those needs.

Server-Sent Events (SSE) push updates from the server to the browser over a single open connection. One-way only — server to client. Simple and well-supported.

```
@GetMapping(value = "/stream", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
public SseEmitter stream() {
    SseEmitter emitter = new SseEmitter();
    new Thread(() -> {
        try {
            for (int i = 0; i < 10; i++) {
                emitter.send("event " + i);
                Thread.sleep(1000);
            }
            emitter.complete();
        } catch (Exception e) { emitter.completeWithError(e); }
    }).start();
    return emitter;
}
```

The browser opens an `EventSource` and receives the messages.

WebSockets go further — full-duplex communication. Both client and server can send messages whenever they like. Add the websocket starter, configure a message broker, and write a controller:

```
@Configuration
@EnableWebSocketMessageBroker
public class WsConfig implements WebSocketMessageBrokerConfigurer {

    @Override
    public void configureMessageBroker(MessageBrokerRegistry config) {
        config.enableSimpleBroker("/topic");
        config.setApplicationDestinationPrefixes("/app");
    }

    @Override
    public void registerStompEndpoints(StompEndpointRegistry r) {
        r.addEndpoint("/ws").withSockJS();
    }
}

@Controller
public class ChatController {
```

```
@MessageMapping("/chat")
@SendTo("/topic/messages")
public String send(String msg) {
    return msg;
}
```

A client sends a message to `/app/chat`, and every subscriber to `/topic/messages` receives it. This is the foundation for chat applications, multiplayer games, and live collaboration tools.

Chapter 33: Reactive Programming with WebFlux

The default Spring MVC stack uses one thread per request. While the thread waits for the database to reply, it is blocked — useless to anyone else. For applications that handle thousands of concurrent requests with slow back-ends, this can become expensive in memory.

Spring WebFlux is a parallel stack that does not block. One thread handles many requests concurrently by switching between them whenever any of them is waiting. Add `spring-boot-starter-webflux` instead of `spring-boot-starter-web`, and your controllers return `Mono` (0-or-1 value) and `Flux` (0-or-many) instead of regular objects:

```
@RestController
public class ProductController {

    @GetMapping("/products")
    public Flux<Product> all() {
        return repo.findAll();
    }

    @GetMapping("/products/{id}")
    public Mono<Product> one(@PathVariable Long id) {
        return repo.findById(id);
    }
}
```

For the database too, you need a non-blocking driver — that means **R2DBC** (reactive relational database connectivity) instead of plain JDBC.

WebFlux is powerful but adds mental overhead — debugging reactive code is harder than debugging plain code. Use it when you genuinely have a scaling problem, not by default.

A simpler alternative arrived with **Java 21** — virtual threads. You write blocking code as usual, but the JVM gives each request a virtual thread that is extremely cheap. Spring Boot 3.2+ supports virtual threads with one property:

```
spring.threads.virtual.enabled=true
```

Many teams that previously considered WebFlux are now staying with plain MVC plus virtual threads.

Chapter 34: Spring Batch and GraphQL

Spring Batch is the toolkit for big data jobs — reading 10 million rows from a file, transforming each, and writing them to a database. A batch job has steps. Each step has a reader, a processor, and a writer. Spring Batch handles chunking (process N items at a time), restarts (resume from where you crashed), and parallel execution.

```
@Bean
public Step step(JobRepository jr, PlatformTransactionManager tm,
                ItemReader<User> r, ItemProcessor<User, User> p, ItemWriter<User>
w) {
    return new StepBuilder("step1", jr)
        .<User, User>chunk(100, tm)
        .reader(r).processor(p).writer(w)
        .build();
}
```

Real uses include nightly invoice runs, bulk imports, and analytical ETL pipelines.

GraphQL is a different style of API. Instead of many endpoints, you have one — and the client describes exactly what it wants in a query. Imagine asking for a user and their five most recent orders, but only certain fields of each. With REST, you would call multiple endpoints. With GraphQL, one query returns exactly what you asked for.

Add `spring-boot-starter-graphql`, define a schema in `src/main/resources/graphql/schema.graphqls`:

```
type Query {
  productById(id: ID!): Product
  products: [Product]
}
type Product {
  id: ID
  name: String
  price: Float
}
```

Write the controller:

```
@Controller
public class ProductGraphController {

    @QueryMapping
    public Product productById(@Argument Long id) {
        return repo.findById(id).orElseThrow();
    }

    @QueryMapping
    public List<Product> products() {
        return repo.findAll();
    }
}
```

GraphQL is excellent when frontend teams need flexibility — mobile, web, and TV apps all asking the same backend for different shapes of data.

Chapter 35: Messaging with Kafka and RabbitMQ

When microservices grow beyond a few, calling each other over HTTP becomes brittle. A failure in one service cascades through the call chain. The cure is **asynchronous messaging** — services publish events to a broker, and other services consume them on their own schedule.

Apache Kafka is the most popular event broker today. It is fast, durable, and good at high volumes.

Producing a message:

```
@Autowired
KafkaTemplate<String, String> kafka;

kafka.send("orders", "Order #123 placed");
```

Consuming one:

```
@KafkaListener(topics = "orders", groupId = "billing")
public void onOrder(String msg) {
    log.info("Received: {}", msg);
}
```

In an e-commerce setup, the order service publishes `OrderPlaced` to Kafka. The billing service consumes it and charges the card. The email service consumes it and sends a confirmation. The inventory service consumes it and deducts stock. None of them know about the others. Each can scale, fail, and restart independently.

RabbitMQ is the other popular broker, with a different model — exchanges and queues rather than topics and partitions. It excels at smaller volumes and more complex routing patterns. Use either — both integrate

beautifully with Spring Boot.

A pattern to remember when combining database writes with message publishing is the **outbox pattern**. If your service saves an order to its database AND publishes a Kafka event, what happens if the database commits but the broker is down? You have an inconsistent system. The outbox pattern: in the same transaction, insert a row into a database "outbox" table. A background job reads the outbox and publishes to Kafka, marking each as sent. Now the database and broker are guaranteed consistent.

Chapter 36: Microservices with Spring Cloud

A microservice architecture splits a monolithic application into many small services, each with its own database, deployed independently. Spring Cloud is the umbrella project that provides the infrastructure pieces.

Service discovery with **Eureka** acts like a phone book. Each service registers itself at startup. When the order service wants to call the user service, it asks Eureka, "where is user-service?" Eureka returns an address. If the user service moves or scales, Eureka knows.

```
@SpringBootApplication
@EnableEurekaServer
public class EurekaApp {
    public static void main(String[] a) { SpringApplication.run(EurekaApp.class,
a); }
}
```

Client services add `eureka.client.service-url.defaultZone=http://localhost:8761/eureka` and `spring.application.name=order-service` to register.

Config Server centralizes configuration. Instead of every service carrying its own `application.properties`, the config server pulls from a Git repository. Change a property in Git, push, and tell services to refresh.

Spring Cloud Gateway is the front door. All external traffic enters through the gateway, which routes to the right service, handles authentication, applies rate limits, and adds metrics.

```
spring:
  cloud:
    gateway:
      routes:
        - id: product
          uri: lb://product-service
          predicates:
            - Path=/api/products/**
        - id: order
          uri: lb://order-service
          predicates:
            - Path=/api/orders/**
```


The `lb://` prefix tells the gateway to load-balance through Eureka.

When services call each other, things go wrong — timeouts, errors, slowness. **Resilience4j** provides patterns to cope:

```
@CircuitBreaker(name = "paymentService", fallbackMethod = "fallback")
public PaymentResponse pay(Order o) { ... }

public PaymentResponse fallback(Order o, Throwable t) {
    return new PaymentResponse("PENDING");
}
```

A circuit breaker watches calls to the payment service. If too many fail, the breaker "opens" — calls go straight to the fallback method without even trying. After a cooldown, it lets a few calls through to test if the service is healthy again. This prevents one failing service from dragging down the entire system.

Other Resilience4j patterns include `@Retry`, `@RateLimiter`, `@Bulkhead`, and `@TimeLimiter`. Each tackles a different failure mode.

To trace a single user's request as it bounces across many services, use **distributed tracing**. Each request gets a unique trace ID. Every service stamps its logs with that ID. You feed all the logs into a tool like Zipkin or Jaeger, and you see exactly which service was the bottleneck. Micrometer Tracing is the modern Spring Boot tool for this.

Chapter 37: The Latest Spring Boot 3 Features

Spring Boot 3, released in late 2022, brought important advances. The framework moved to Java 17 as a minimum. Jakarta EE replaced the old `javax.*` package names with `jakarta.*`. Native image support became first-class.

GraalVM native images compile your Spring Boot app to a native binary. It starts in 50 milliseconds, uses a fraction of the memory, and is perfect for serverless platforms where every cold start counts:

```
mvn -Pnative native:compile
```

Run the resulting binary directly — no JVM needed.

Observability got a major overhaul. The combination of Micrometer Metrics, Micrometer Tracing, and Actuator gives you a unified picture — metrics, traces, and health all in one place. Add the right Micrometer registry (Prometheus, Datadog, New Relic), and observability data flows out automatically.

Virtual threads, introduced as a preview in Java 19 and stable in Java 21, let you handle millions of concurrent requests without the complexity of reactive programming. As mentioned earlier, enable them with a single property in Spring Boot 3.2+.

Spring Boot 3.1 brought Docker Compose support, Testcontainers `@ServiceConnection`, SSL bundles for reusable certificate configurations, and the new `RestClient` HTTP client. **Spring Boot 3.2** brought stable

virtual thread support and the [HttpExchange](#) interface for declarative HTTP clients without Feign.

The framework continues to move forward. Stay close to the [Spring Boot release notes](#) — each minor version brings useful additions.

APPENDIX

Appendix A — Common Errors and How to Fix Them

When your application fails, the error message usually points the way. A few error messages come up so often that recognizing them by sight saves hours.

No qualifying bean of type ... means Spring could not find a bean of the type you asked for. Either the class is missing the right stereotype annotation ([@Service](#), [@Component](#)), or it sits in a package that is not being scanned. Check that your main application class is in a package above all your other code.

Failed to configure DataSource: 'url' attribute is not specified means your database settings are missing or wrong. Verify that [spring.datasource.url](#), [username](#), and [password](#) are set, and that the driver dependency is in your [pom.xml](#).

LazyInitializationException: could not initialize proxy - no Session means you tried to access a lazily-loaded JPA relationship outside a transaction. Either wrap the call in a [@Transactional](#) method, or use [JOIN FETCH](#) to load the relationship up front.

Whitelabel Error Page simply means Spring could not find a controller mapping for the URL you visited. Check the URL and the controller method's mapping.

403 Forbidden means Spring Security blocked the request because the user lacks the required role. **401 Unauthorized** means there was no authentication at all — you forgot to send the Bearer token.

Port 8080 already in use means another process — possibly another running copy of your own app — is holding the port. Either stop the other process or set [server.port=8081](#).

When Lombok annotations show as red errors in Eclipse, you forgot to install the Lombok agent — re-run [java -jar lombok.jar](#) and point it at Eclipse.

Appendix B — The Bookstore Project Blueprint

The complete bookstore project we have been building throughout this book has the following structure:

```
com.example.bookstore
├─ BookstoreApplication.java
├─ config/           (SecurityConfig, CorsConfig, OpenApiConfig)
├─ security/         (JwtFilter, JwtService, UserDetailsServiceImpl)
├─ user/
│   ├─ User.java
│   ├─ UserRepository.java
│   └─ UserService.java
```

```
|   |   | AuthController.java
|   |   |   |   | dto/
|   |   | book/
|   |   |   |   | Book.java
|   |   |   |   | BookRepository.java
|   |   |   |   | BookService.java
|   |   |   |   | BookController.java
|   |   |   |   |   | dto/
|   |   | order/
|   |   |   |   | Order.java
|   |   |   |   | OrderItem.java
|   |   |   |   | OrderRepository.java
|   |   |   |   | OrderService.java
|   |   |   |   | OrderController.java
|   |   | exception/
|   |   |   |   | ResourceNotFoundException.java
|   |   |   |   | ApiError.java
|   |   |   |   | GlobalExceptionHandler.java
```

The endpoints:

```
POST   /auth/register
POST   /auth/login           returns JWT
GET     /api/books?page=0&size=10
GET     /api/books/{id}
POST    /api/books           ADMIN only
PUT     /api/books/{id}      ADMIN only
DELETE  /api/books/{id}      ADMIN only
POST    /api/orders          USER only
GET     /api/orders/me       USER only
GET     /actuator/health
GET     /swagger-ui.html
```

Build the project in this order:

1. Create the project on start.spring.io with the dependencies listed in Chapter 4.
2. Configure the database in `application.properties`. Start with H2 for simplicity.
3. Build the `Book` entity, repository, service, and controller. Test in Postman.
4. Add validation rules and a global exception handler.
5. Add the `User` entity, registration endpoint, and BCrypt password hashing.
6. Add Spring Security and a JWT filter. Add a login endpoint that returns a token.
7. Protect endpoints with role rules (USER, ADMIN).
8. Add the `Order` flow with `@Transactional`, deducting book stock and saving the order in one atomic operation.
9. Add Swagger and Actuator.
10. Add caching to the book list endpoint.
11. Write unit tests for services and integration tests for controllers.
12. Dockerize the application and run with docker-compose alongside a MySQL container.

13. Deploy to a cloud platform of your choice.

When you have done all thirteen steps, you have built a production-style backend project end to end.

Appendix C — All Annotations Reference Card

Bean stereotypes: `@Component`, `@Service`, `@Repository`, `@Controller`, `@RestController`

Configuration: `@Configuration`, `@Bean`, `@Value`, `@PropertySource`, `@Import`, `@Profile`, `@ConfigurationProperties`, `@EnableConfigurationProperties`

Dependency injection: `@Autowired`, `@Qualifier`, `@Primary`

Conditional beans: `@ConditionalOnClass`, `@ConditionalOnMissingBean`, `@ConditionalOnProperty`, `@ConditionalOnExpression`

Web — request mapping: `@RequestMapping`, `@GetMapping`, `@PostMapping`, `@PutMapping`, `@PatchMapping`, `@DeleteMapping`

Web — parameters: `@RequestBody`, `@ResponseBody`, `@PathVariable`, `@RequestParam`, `@RequestHeader`, `@CookieValue`, `@ModelAttribute`, `@InitBinder`

Web — response: `@ResponseStatus`, `@CrossOrigin`

JPA — entity: `@Entity`, `@Table`, `@Id`, `@GeneratedValue`, `@Column`, `@Enumerated`, `@Lob`, `@Transient`, `@Embedded`, `@Embeddable`, `@EmbeddedId`, `@IdClass`

JPA — relationships: `@OneToOne`, `@OneToMany`, `@ManyToOne`, `@ManyToMany`, `@JoinColumn`, `@JoinTable`

JPA — queries: `@Query`, `@Modifying`, `@Param`, `@EntityGraph`, `@Procedure`, `@NamedQuery`

JPA — auditing & locking: `@CreatedDate`, `@LastModifiedDate`, `@CreatedBy`, `@LastModifiedBy`, `@EntityListeners`, `@Version`, `@Lock`

JPA — lifecycle callbacks: `@PrePersist`, `@PostPersist`, `@PreUpdate`, `@PostUpdate`, `@PreRemove`, `@PostRemove`, `@PostLoad`

Validation: `@Valid`, `@NotNull`, `@NotBlank`, `@NotEmpty`, `@Size`, `@Min`, `@Max`, `@Email`, `@Pattern`, `@Positive`, `@Negative`, `@Past`, `@Future`, `@AssertTrue`, `@AssertFalse`

Transactions: `@Transactional`

Security: `@EnableWebSecurity`, `@EnableMethodSecurity`, `@PreAuthorize`, `@PostAuthorize`, `@PreFilter`, `@PostFilter`, `@RolesAllowed`, `@Secured`, `@WithMockUser`

Caching: `@EnableCaching`, `@Cacheable`, `@CachePut`, `@CacheEvict`, `@Caching`

Scheduling and async: `@EnableScheduling`, `@Scheduled`, `@EnableAsync`, `@Async`, `@SchedulerLock`

AOP: `@Aspect`, `@Pointcut`, `@Before`, `@After`, `@AfterReturning`, `@AfterThrowing`, `@Around`

Events: `@EventListener`, `@TransactionalEventListener`

Testing: `@SpringBootTest`, `@WebMvcTest`, `@DataJpaTest`, `@JsonTest`, `@RestClientTest`, `@JdbcTest`, `@MockBean` (or `@MockitoBean` in newer versions), `@SpyBean`, `@TestConfiguration`, `@ActiveProfiles`, `@DirtiesContext`, `@Sql`

Exception handling: `@ControllerAdvice`, `@RestControllerAdvice`, `@ExceptionHandler`

Application: `@SpringBootApplication`, `@EnableAutoConfiguration`, `@ComponentScan`, `@EnableFeignClients`, `@EnableEurekaServer`, `@EnableConfigServer`

Appendix D — All Lombok Annotations

`@Getter` and `@Setter` generate getters and setters on the field or class. `@ToString` generates `toString()`. `@EqualsAndHashCode` generates `equals()` and `hashCode()`. `@NoArgsConstructor` creates an empty constructor. `@AllArgsConstructor` creates a constructor with all fields. `@RequiredArgsConstructor` creates a constructor only with `final` fields — perfect for Spring's constructor injection. `@Data` is the all-in-one — `@Getter`, `@Setter`, `@ToString`, `@EqualsAndHashCode`, `@RequiredArgsConstructor`. `@Value` is the immutable cousin of `@Data` — all fields are `final`, no setters. `@Builder` generates a fluent builder pattern. `@Slf4j` adds a logger field named `log`. `@SneakyThrows` suppresses checked exceptions. `@Cleanup` auto-closes resources. `@With` creates copy-on-write methods.

A typical Spring Boot service uses `@Service`, `@RequiredArgsConstructor`, and `@Slf4j` together. A typical entity uses `@Entity`, `@Data`, `@NoArgsConstructor`, `@AllArgsConstructor`. A typical DTO uses Java records — but if you cannot, `@Value` is the Lombok equivalent.

Appendix E — Where Each Concept Is Used in Real Projects

Every concept in this book has a real-world home. The `@RestController` annotation lives in every REST API. The `@Service` annotation marks the business logic class — the brain of the application. JPA with Hibernate is how Java talks to MySQL and PostgreSQL. The `@Transactional` annotation guards banking transfers, order placements, anything where consistency matters. Validation guards every API input. Global exception handlers belong in every production API. Spring Security handles login and permissions. JWT is the standard for mobile apps and single-page apps. BCrypt is how you store passwords. Actuator's health endpoint is what Kubernetes pings to know whether your app is alive. Caching speeds up product catalogs and reference data. Scheduling sends daily reports and cleans up expired data. Async work sends order confirmation emails without making the user wait. Docker is how every modern application is deployed. Eureka and Spring Cloud Gateway power microservice architectures. Circuit breakers protect your system when external services fail. Swagger is how backend teams hand off APIs to frontend and mobile teams. AOP is how you add cross-cutting concerns like audit logs without polluting business code. Flyway is how you migrate database schemas safely in production. Kafka and RabbitMQ are how microservices communicate asynchronously.

When in doubt about whether to learn a topic deeply, ask yourself: would I use this in a real project? If yes, learn it now. If maybe, bookmark it for later.

Appendix F — Final Checklist

When you can tick every box below, you can build backend projects on your own with confidence.

- I understand IoC, dependency injection, and how the Spring container works.
- I can structure a project into controller, service, repository, and entity layers.
- I know what the common annotations mean without looking them up.
- I can build CRUD REST APIs with proper HTTP methods and status codes.
- I use DTOs to separate API contracts from database entities.
- I validate every input with bean validation and `@Valid`.
- I have a global exception handler that returns clean error JSON.
- I can write JPA entities with all relationship types.
- I use `@Transactional` correctly on service methods.
- I can write derived queries, custom `@Query` queries, and paginated results.
- I can secure an API with Spring Security and JWT.
- I hash passwords with BCrypt.
- I can write unit tests with Mockito and integration tests with MockMvc.
- I use Actuator for health checks and metrics.
- I configure logging properly with log levels and structured output.
- I add caching, scheduling, and async behaviors where appropriate.
- I call external APIs with `RestClient` or `WebClient` or OpenFeign.
- I use profiles and environment variables to manage configuration across environments.
- I containerize my app with Docker and run it locally with docker-compose.
- I have built the bookstore project end to end.

When all boxes are ticked, stop reading. Go build.

Final Words

You have reached the end of the book. That alone deserves congratulations — long-form learning is rare in a world of quick tutorials. But reading is only half the journey. The other half is building.

Pick a small idea — a personal recipe app, a task tracker, a portfolio backend — and build it. You will get stuck. You will hit errors you have never seen before. Each error you solve becomes a permanent skill.

When you are stuck, search for your error message. The Spring community is enormous and someone else has almost certainly hit the same problem. The Baeldung website is a treasure trove of Spring tutorials. The official Spring Boot reference is dense but accurate when you need depth.

Spring Boot will keep evolving. New versions arrive every six months. Stay curious. Read the release notes. Try the new features in a side project before adopting them at work.

May your applications start fast, your tests pass, and your servers stay up at 3 AM.

Happy coding.